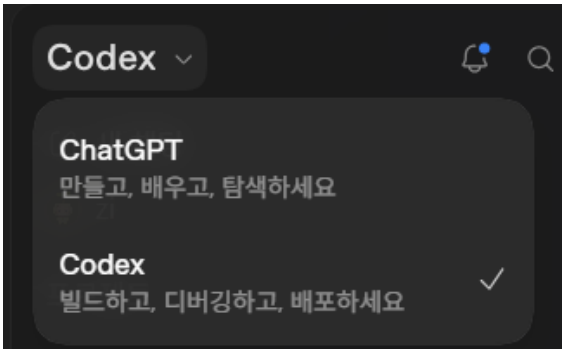
codex에서 chatGPT 사용하기 - 1

💡 목표 : Codex가 ChatGPT를 자체적으로 판단하여, 하위 에이전트로 부리게 만들고 싶다!

왜? 굳이? > Codex 작업자와 일반 Chat은 다르니까.



생각: 구독 요금제 'Pro'의 *일반 Chat GPT-5.6 Sol 무제한* 혜택을 활용한다. **ChatGPT Work는 Codex와 사용량을 공유**하므로 이 계산에서 **제외**한다. pro(요금제)가 아니어도 토큰 단가는 codex에 비해 현저히 저렴하다.

그렇다면, Pro 일반 Chat 5.6 Sol을 활용한 개발 프로젝트 절감 예상은?

단, 모든 개발 작업을 넘길 수는 없다. 저장소 탐색, 파일 수정, 실행, 테스트, 통합은 Codex가 맡고, **설계·추론·검토·초안처럼 파일 없이 설명할 수 있는 작업만** Chat에 맡긴다고 가정했을 때.

계산식은 아래과 같이 설계.

> **프로젝트 Codex 사용량 절감률 = 위임 가능한 작업이 차지하던 Codex 사용량 비율 × (1 − 전달·검수에 남는 비율)**

프로젝트 구성의 **가정**	기존 Codex 사용량 중 Chat에 넘길 수 있는 비율	전달·검수에 다시 쓰는 비율	**예상 Codex 사용량 감소**
코드 수정·테스트 중심	30%	위임분의 20%	**24%**
구현과 설계·검토가 비슷한 프로젝트	50%	위임분의 15%	**42.5%**

프로젝트 구성의 **가정**	기존 Codex 사용량 중 Chat에 넘길 수 있는 비율	전달·검수에 다시 쓰는 비율	**예상 Codex 사용량 감소**
기획·아키텍처·문서·분석 비중이 큰 프로젝트	70%	위임분의 10%	**63%**
고난도 읽기 작업을 적극적으로 Chat에 배분	80%	위임분의 10%	**72%**

특히 효율적으로 운영한다면? > 어쩌면 설계 지침

일반 Chat에 맡길 일	Codex에 남길 일
아키텍처 대안의 장단점 분석	실제 저장소 구조 확인
요구사항의 모순·누락 찾기	현재 코드와 요구사항 대조
알고리즘·데이터 모델 설계안 여러 개 생성	코드 작성과 통합
전달된 오류 설명을 바탕으로 원인 가설 수립	로그 재현, 디버깅, 테스트
테스트 관점·실패 사례 제안	테스트 작성·실행
설계 문서, 교육 자료, 릴리스 문안 초안	사실 확인과 최종 반영

'내가 생각하는' 가장 효율적인 형태: Codex가 chat 기능이 더 효율적이라 판단 → Codex가 chat에 문제와 완료 기준을 짧게 전달 → Chat에서 여러 차례 깊게 추론 → 구조화된 최종 결과 한 번 제출 → Codex가 근거만 검수하고 적용. 단, Codex가 Chat의 매 추론 단계에 계속 끼어들면 절감률이 낮아질 수 있다.

첫 번째 해결과제

Codex는 자체 하위 작업자를 사용해 일을 나눌 수 있으나, **일반 ChatGPT 는 Codex의 하위 작업자로 배정할 수 없다.**

> 즉, Codex가 일반 Chat에 메시지를 보냈다고 해서, 그 대화의 완료를 Codex가 자동으로 기다리거나, 그 결과를 원래 작업에서 받을 수 있는 것은 아니다.

하지만 내가 원하는 방향

예를 들어 Codex가 기능 구현을 총괄하면서 일반 ChatGPT에 "이 명세에서 빠진 조건을 찾아 달라"고 맡긴다고 가정하자. **실제 코덱스와 chat 간에 협업이 성립되려면 다음 조건들을 만족해야 한다.**

1. **정확한 대상에게 전달:** 어느 계정의 어느 일반 Chat 대화인지 확인한다.
2. **작업 맥락 유지:** 요청의 작업 ID, 기준 코드 버전, 검수 기준을 함께 전달한다.

3. **결과 회수:** Chat의 답변을 사람이 복사해서 옮기지 않아도 원래 작업이 찾을 수 있어야 한다.
4. **원래 작업에 알림:** 결과가 도착했을 때 엉뚱한 Codex 작업이 아니라 요청을 보낸 작업에 알려야 한다.
5. **검수:** 결과가 도착한 것과 Codex가 그 결과를 받아들인 것은 별도로 기록한다.

이 다섯 단계를 **하나의 추적 가능한 흐름**으로 만들어야 한다.

결국 MCP

1. **작업 전달:** Codex가 기존 일반 ChatGPT 대화에 작업 메시지를 보낸다.
2. **결과 제출:** ChatGPT가 답을 만든 뒤, 연결된 `submit_result` 도구를 호출해 결과를 로컬에 제출한다.
3. **결과 검수**: 로컬 알림 코드는 **결과가 도착했다는 사실**을 기존 Codex 세션에 알려준다. Codex는 저장된 결과를 읽고 검수한다.

```
① 작업 전달
[원래 Codex 작업] ──메시지 전송──▶ [지정된 일반 ChatGPT 대화]

② 결과 제출
[일반 ChatGPT 대화] ──MCP 도구 submit_result 호출──▶ [로컬 결과 수신기]
                                          │
                                          ▼
                                   [결과 파일 저장]

③ 도착 알림과 검수
[결과 파일 저장] ──알림 코드·codex queue──▶ [원래 Codex 작업]
                                          │
                                          ▼
                                [결과 확인·채택 또는 거절]
```

flow 예시

```
"이 명세에서 빠진 예외 조건을 찾아 목록으로 제출해 줘. 파일은 수정하지 마."
```

① **Codex에서 작업 요청.** Codex는 확인된 기존 ChatGPT 대화에 이 요청과 필요한 명세 발췌문을 메시지로 보낸다. 이때 사용하는 것은 Codex 앱의 기존 대화 전송 도구인 `send_message_to_thread`다. **ChatGPT 모델 API를 새로 호출하거나 Codex 하위 작업자를 생성하는 방식이 아니다.**

② **ChatGPT가 결과를 제출.** ChatGPT가 예외 조건 목록을 작성하면, 연결된 별도의 MCP 도구 `submit_result`를 호출하도록 요청받는다. 이 호출의 방향은 **ChatGPT → 로컬 MCP 수신기**. 수신기는 작업 ID와 권한 등을 확인한 뒤 결과를 파일로 저장한다. ChatGPT가 답변만 작성하고 도구를 호출하지 않으면, MCP 결과 제출은 일어나지 않는다.

③ **상위 에이전트 Codex가 작업 알림 수신.** 저장이 끝나면 알림 코드가 `codex queue`를 이용해 *처음 요청을 보낸 Codex 작업*에 "결과가 저장됐다"는 메시지를 보낸다. 알림은 결과 본문을 무조건 채택하라는 지시는 아니다.

④ **Codex가 검수.** Codex는 저장된 결과를 열어 해당 요청의 결과인지, 기준으로 삼은 파일 버전이 맞는지, 내용이 타당한지 확인한다. 이 검수가 끝나야 결과를 채택했다고 말할 수 있다.

기대 효과

- **codex와 chat 간, 불필요한 human in to loop 삭제:** 사람이 Chat 답변을 복사해 원래 Codex 대화에 붙여 넣는 단계 탈피
- **작업의 연속성:** 요청을 보낸 Codex 작업이 결과의 도착과 검수를 이어서 처리하는 구조
- **검수 가능성:** 어떤 요청에 대한 결과인지, 어느 코드 버전을 기준으로 작성했는지, 최종 채택했는지를 구분하여 작업의 연속성 및 신뢰성 확보
- **역할 분리:** 파일 시스템에 접근할 필요가 없는 작업, 그 중에서도 고난도 작업(설계 및 디자인)을 Chat에 할애하고, 검토와 저장소 변경, 마지막 최종 결정를 Codex가 담당한다.

생각해볼 점: 파일이 필요한 작업도 확장할 여지는 있다

현재 시험용 MCP는 Chat의 **결과 제출**만 지원한다. 향후 범위를 제한한 *읽기 전용 파일 검색·발췌 도구*가 생기면, Chat이 필요한 코드 조각을 가져와 **일부 저장소 분석도 일반 Chat 사용 가능량에서 처리**할 수 있다. 그러면 위임 가능한 비율은 올라갈 수 있다.

Chat이 읽은 파일 내용도 Chat 측에서는 토큰으로 처리되며, 변경·테스트·최종 검수는 계속 Codex에 남는다. 현재 설계는 직접 저장소 쓰기를 허용하지 않고 있다.